MINISTRY OF EDUCATION AND TRAINING

LOGO

FPT UNIVERSITY

Capstone Project Document

SAP Bug Tracking Management System (ZBUG_WS)

Group ZBUG	
Group Members	DEV-089 — Manager | SE11, SE38, SE80, SE93 DEV-061 — Developer | ALV Grid & SmartForms DEV-118 — Tester | Bug Management, Testing
Supervisor	
Ext Supervisor	

— Hanoi, April 2026 —

Table of Contents

Acknowledgement 1
Definition and Acronyms 2
I. Project Introduction 3
1. Overview 3
1.1 Project Information 3
1.2 Project Team 3
2. Product Background 3
3. Existing Solutions 4
4. Business Opportunity 4
5. Software Product Vision 5
6. Project Scope & Limitations 5
In Scope 5
Out of Scope 6
Constraints & Limitations 6
II. Project Management Plan 7
1. Overview 7
1.1 Scope & Estimation 7
1.2 Project Objectives 8
1.3 Project Risks 8
2. Management Approach 10
2.1 Project Process 10
2.2 Quality Management 10
2.3 Training Plan 11
3. Project Deliverables 11
4. Responsibility Assignments 12
5. Project Communications 13
6. Configuration Management 13
6.1 Document Management 13
6.2 Source Code Structure 13
6.3 Tools & Infrastructures 14
III. Software Requirement Specification 15
1. Requirement Overview 15
1.1 Context Diagram 15

1.2 User Requirements 15
1.3 System Functionalities 16
2. Functional Specifications 17
2.1 Bug Management 17
2.2 Project Management 18
3. UI Requirements 19
3.1 Project Search (Screen 0410 — Initial Screen) 19
3.2 Bug List (Screen 0200) 20
3.3 Bug Detail (Screen 0300 — Tab Strip) 20
3.4 Status Transition Popup (Screen 0370 — v5.0 New) 21
4. Non-Functional Requirements 21
4.1 External Interfaces 21
4.2 Quality Attributes 22
5. Requirement Appendix 22
5.1 Business Rules 22
5.2 Common Requirements 23
5.3 Application Messages List 23
5.4 Other Requirements 24
IV. Software Design Description 25
1. System Design 25
1.1 System Architecture 25
1.2 Package Diagram 25
2. Database Design 25
Table: ZBUG_TRACKER — 29 fields 25
Table: ZBUG_USERS — 12 fields 26
Table: ZBUG_PROJECT — 16 fields 27
Table: ZBUG_USER_PROJEC — 10 fields 27
Table: ZBUG_HISTORY — 10 fields 27
3. Detailed Design 28
3.1 Bug Creation (UC-05) 28
3.2 Status Transition (UC-08) 28
3.3 Auto-Assign Engine 29
3.4 Role-Based Screen Control (LOOP AT SCREEN) 29
V. Software Testing Documentation 31
1. Scope of Testing 31

1.1 Target-of-Test Features 31
1.2 Test Levels 32
1.3 Constraints and Assumptions 32
2. Test Strategy 33
2.1 Testing Types 33
2.2 Test Levels 34
2.3 Supporting Tools 34
3. Test Plan 35
3.1 Human Resources 35
3.2 Test Environment 35
3.3 Test Milestones 36
4. Test Cases 37
5. Test Reports 38
5.1 UAT Round 1 — Phase E Results (11–13 April 2026) 38
5.2 QC Full Test Run — Phase F (Planned) 39
VI. Release Package & User Guides 41
1. Deliverable Package 41
2. Installation Guides 43
2.1 System Requirements 43
2.2 Installation Steps 43
3. User Manual 46
3.1 Overview 46
3.2 Workflow 1 — Bug Reporting (Tester Role) 47
3.3 Workflow 2 — Bug Resolution (Developer Role) 48
3.4 Workflow 3 — Project Management (Manager Role) 49

Acknowledgement

This report documents the design, implementation, and testing of the **SAP Bug Tracking Management System** — project code **ZBUG_WS** — developed on SAP S40 (Client 324) using ABAP 7.70.

The team acknowledges the opportunity to work on a real-world enterprise system using professional SAP development tools: SE11, SE38, SE51, SE41, SE80, SE93, SMARTFORMS, and SCOT.

The project delivered a full-featured, role-based bug tracking system built as a Module Pool (Type M) ABAP program — surpassing the reference system (`ZPG_BUGTRACKING_*`) in architecture, automation, auditability, and UI quality across 6 development phases (A through F).

Definition and Acronyms

Acronym	Definition
ABAP	Advanced Business Application Programming — SAP's proprietary language
ALV	ABAP List Viewer — standard SAP grid component
BCS	Business Communication Services — SAP email API (CL_BCS)
BUG_ID	Unique bug identifier, format BUG0000001, auto-generated via ZNRO_BUG
CRUD	Create, Read, Update, Delete — basic data operations
DEV	Developer — role responsible for fixing bugs
ERD	Entity Relationship Diagram
GOS	Generic Object Services — SAP file attachment framework
GUI	Graphical User Interface
MGR	Manager — role with full system access
PAI	Process After Input — screen event after user action
PBO	Process Before Output — screen event before screen display
PDF	Portable Document Format — SmartForm print output
SAP	Systems, Applications and Products in Data Processing
SDD	Software Design Description
SMTP	Simple Mail Transfer Protocol
SPMP	Software Project Management Plan
SRS	Software Requirement Specification
SE11	SAP Data Dictionary — table/domain/data element creation
SE41	Menu Painter — GUI Status and Title Bar editor
SE51	Screen Painter — Dynpro screen layout editor
SE80	Object Navigator — main ABAP Workbench
SE93	Transaction Maintenance — T-code editor
T-Code	Transaction Code — SAP program shortcut (e.g., ZBUG_WS)
UAT	User Acceptance Test
UC	Use Case
ZBUGTRACK	SAP Development Package containing all `ZBUG_*` objects
ZBUG_WS	Entry T-Code for the Bug Tracking system

I. Project Introduction

1. Overview

1.1 Project Information

Project Name	**SAP Bug Tracking Management System**
Project Code	`ZBUG_WS`
Program	`Z_BUG_WORKSPACE_MP` (Module Pool, Type M)
SAP Package	`ZBUGTRACK`
SAP System	S40 | Client 324 | ABAP 7.70
T-code	`ZBUG_WS`
Group	Group ZBUG
Version	v5.0 (Phase F — Enhancement)
Report Date	April 2026

1.2 Project Team

Account	Role	System Expertise	Responsibilities
`DEV-089`	Manager (M)	SE11, SE38, SE80, SE93	Database hardening, project management, full system access, reporting
`DEV-061`	Developer (D)	ALV Grid, SmartForms	Bug fix implementation, code development, evidence upload (fix_report)
`DEV-118`	Tester (T)	Bug Management, QA	Bug reporting, UAT execution, fix verification, evidence upload (confirm_report)

2. Product Background

SAP ERP implementations involve simultaneous development across multiple modules (MM, SD, FI, CO, etc.). Without a native, structured tracking tool, bug reports are scattered across spreadsheets, emails, and instant messages. This makes it difficult to enforce accountability, track workload distribution, or maintain an auditable history of changes.

This project was initiated by FPT University to build a **centralized bug tracking system** running natively inside SAP, accessible via a single T-code (`ZBUG_WS`). The system was developed on SAP system S40 (Client 324) using ABAP 7.70, from March to April 2026, across six structured phases (A through F).

The v5.0 enhancement (Phase F) introduced the following major improvements over the initial v4.2 release:

- A 10-state bug lifecycle replacing the previous 9-state model
- A dedicated status transition popup (Screen 0370) that enforces role-based transition rules
- An auto-assign engine for both developers and testers (based on workload and SAP module)
- A bug search engine with cross-field filtering (Screens 0210/0220)
- A real-time dashboard header on the bug list screen

3. Existing Solutions

The reference system `ZPG_BUGTRACKING_MAIN` / `ZPG_BUGTRACKING_DETAIL` provides a basic bug list and detail screen using an executable ABAP program. While it demonstrates the core concept, it has significant gaps compared to `ZBUG_WS`:

Feature	**ZPG Reference**	**ZBUG_WS**
Module Pool (Type M) architecture	No	Yes
Auto-assign developer by module/workload	No	Yes
Centralized permission system (FM-based)	No	Yes
Full audit history log (`ZBUG_HISTORY`)	No	Yes
Email notification via CL_BCS	No	Yes
SmartForm PDF print output	No	Yes
Project management module	No	Yes
10-state lifecycle with transition popup	No	Yes (v5.0)
Dashboard metrics header (Screen 0200)	No	Yes (v5.0)
Bug search engine (Screen 0210/0220)	No	Yes (v5.0)

4. Business Opportunity

SAP development teams in large organizations face a critical gap: no built-in tool for tracking bugs assigned to individual developers, with role-based visibility and workflow enforcement. The absence of such a tool leads to:

- Bugs being missed, duplicated, or informally tracked outside the system
- Developers receiving unclear assignments without priority or module context
- Managers unable to view real-time workload distribution or project health
- No structured evidence of bug reports, fix confirmations, or test verifications

`ZBUG_WS` addresses all of these by providing a **zero-external-dependency** system embedded directly within SAP. All data is stored in ABAP tables (`ZBUG_TRACKER`, `ZBUG_PROJECT`, etc.), all notifications use SAP's built-in email (SCOT / CL_BCS), and all UI is native SAP GUI (Module Pool). No additional licensing, infrastructure, or integration is required.

This makes it immediately deployable on any SAP ECC or S/4HANA environment supporting ABAP 7.70+.

5. Software Product Vision

For SAP development teams that need structured bug management, `ZBUG_WS` is a Module Pool application providing end-to-end bug tracking with role-based access control, automated assignment, and immutable audit trails. Unlike spreadsheet-based tools or external systems, `ZBUG_WS` runs natively on the SAP platform, requires no additional licensing, and enforces workflow rules that cannot be bypassed by any user role.

Key outcomes delivered:

- Every bug follows a defined 10-state lifecycle; transitions are enforced per role via a popup (Screen 0370)
- Auto-assign engine selects the developer with the lowest workload in the correct SAP module
- All status changes, assignments, and evidence uploads are logged in `ZBUG_HISTORY`
- Managers see real-time metrics (by status, priority, module) on every bug list screen
- Three evidence templates (Bug Report, Fix Report, Confirm Report) are downloadable from SMW0 and uploaded directly from the bug detail screen

6. Project Scope & Limitations

In Scope

- **Bug lifecycle management:** 10-state lifecycle (New, Waiting, Assigned, In Progress, Pending, Fixed, Final Testing, Resolved, Rejected, Closed-legacy) with transition rules per role

- **Project management:** Create / Edit / Delete projects, user-project assignment with per-project roles
- **Role-based access control:** Manager / Developer / Tester, enforced via SAP screen groups (`EDT`, `FNC`, `TST`, `DEV`, `STS`, `BID`, `PRJ`)
- **Auto-assign system:** developer assignment on bug creation (Phase A), tester assignment on fix completion (Phase B)
- **Evidence management:** upload / download of three template-based files per bug (`ZBUG_EVIDENCE` table)
- **Email notification:** automatic send via CL_BCS on create, assign, status change, and reject events
- **Bug search:** cross-field filter by Bug ID, Title, Status, Priority, Module, Date range (Screens 0210/0220)
- **Dashboard header:** live count metrics on Screen 0200 grouped by status, priority, module
- **Audit trail:** all changes logged in `ZBUG_HISTORY` with old/new values and mandatory reason notes

Out of Scope

- Integration with external tools (Jira, ServiceNow, Bugzilla, etc.)
- Web or mobile interface — SAP GUI (Dynpro) only
- Automated test execution framework
- Multi-client or cross-system synchronization
- BI/analytics reporting (BW, Fiori, etc.)

Constraints & Limitations

- Development and testing confined to SAP system S40, Client 324
- ABAP 7.70 syntax required (inline declarations, `SWITCH` expression, string templates, `@` host variables)
- Three fixed demo accounts: `DEV-089` (Manager), `DEV-061` (Developer), `DEV-118` (Tester)
- Evidence files must be `.xlsx` format, maximum 10 MB per file
- Screen 0100 is deprecated (v5.0); entry point is Screen 0410 (Project Search)

II. Project Management Plan

1. Overview

1.1 Scope & Estimation

The following table categorizes each major software function by complexity and estimates the development effort in man-days.

No.	Feature	Description	Simple	Medium	Complex	Total (MD)
1	DB Hardening	Domains, data elements, tables (5 tables, 77 fields), number ranges			1	3
2	Function Modules	`Z_BUG_CREATE`, `Z_BUG_AUTO_ASSIGN`, `Z_BUG_CHECK_PERMISSION`, `Z_BUG_LOG_HISTORY`, `Z_BUG_SEND_EMAIL`			1	5
3	Module Pool UI	Program `Z_BUG_WORKSPACE_MP` + 6 includes, 12 screens, GUI Statuses		1		4
4	Bug List (Screen 0200)	ALV Grid, toolbar, role filter, Dashboard Header (v5.0)		1		3
5	Bug Detail (Screen 0300)	Tab Strip (6 tabs), long text editors, evidence ALV, history ALV			1	4
6	Status Transition Popup	Screen 0370, 10-state lifecycle, role-based field matrix			1	3
7	Project Management	Screens 0400/0410/0500, project CRUD, user-project assignment		1		3

8	Auto-Assign Engine	Phase A (New→Assigned) + Phase B (Fixed→FinalTesting), workload calc			1	3
9	Bug Search Engine	Screens 0210/0220, cross-field filter, result ALV		1		2
10	Email & Evidence	CL_BCS email, `ZBUG_EVIDENCE` upload/ download, template (SMW0)		1		2
11	Testing & Bug Fixes	QC Test Plan (140 cases), UAT (43 cases), 11 UAT bug fixes (v5.0)	1			3
	TOTAL		**1**	**5**	**5**	**35 MD**

1.2 Project Objectives

Overall objective: Deliver a production-ready, role-based bug tracking system running natively on SAP, surpassing the reference system (`ZPG_BUGTRACKING_*`) in all dimensions, within the 10-day sprint window (24 March – 03 April 2026) plus the Phase F enhancement (April 2026).

Specific targets:

Quality	**All 140+ QC test cases executed; pass rate ≥ 90% on UAT Round 2**
Functionality	10-state lifecycle enforced per role; auto-assign success rate 100% when eligible developer/tester exists
Performance	ALV screens load within 3 seconds on S40; no timeout on evidence upload ≤ 10 MB
Effort distribution	Req 10% | Design 15% | Coding 45% | Testing 20% | PM 10%

1.3 Project Risks

No.	Risk	Description	Prob.	Impact	Mitigation
1	SAP system downtime	S40 system unavailable during dev/test windows	Low	High	Schedule critical work in off-peak hours;

					use SE38 offline editing
2	Breaking change: status `6`	v5.0 redefines status `6` from Resolved → Final Testing; migration required	High	High	Write migration script; run only after v5.0 is fully deployed
3	`ZBUG_EVIDENCE` not created	Table may not exist in SAP DB; blocks evidence upload feature	Medium	High	Create table in SE11 before running test cases TC-12
4	SCOT email not configured	CL_BCS email send silently fails if SMTP not set up in SCOT	Medium	Medium	Verify SOST after each email-triggering test; document as known limitation
5	Auto-assign no eligible dev	If no Developer has workload < 5 in the correct module, bug → Waiting	Low	Medium	Insert 30 mock users in `ZBUG_USERS` /

					ZBUG_USER_PROJEC for testing
6	Phase F deploy incomplete	New screens (0410/0370/0210/0220) and GUI Statuses not yet created in SAP	High	High	Follow the step-by-step Phase F Enhancement Guide (F11–F17)

2. Management Approach

2.1 Project Process

The project followed an **incremental waterfall** model divided into six phases, each building upon the previous:

Phase	Name	Deliverables	Status
A	Database Hardening	5 custom tables, domains, data elements, number ranges (`ZNRO_BUG`)	Done
B	Business Logic (FMs)	6 Function Modules: Create, AutoAssign, Permission, History, Email, Evidence	Done
C	Module Pool UI (v4.x)	Program `Z_BUG_WORKSPACE_MP`, 8 screens, GUI Statuses, ALV grids, tab strips	Done
D	Advanced Features (v4.x)	SmartForms, Excel upload/download, message class, F4 helpers, unsaved-changes detection	Done
E	Testing (v4.x)	QC Test Plan (140 cases), UAT (43 cases), UAT Round 1 results (11 bugs found)	Done
F	v5.0 Enhancement	10-state lifecycle, Screen 0370/0410/0210/0220, auto-assign v2, bug search, dashboard	CODE done; deploy pending

2.2 Quality Management

- **Code review:** Each include (`Z_BUG_WS_TOP`, `_F00`, `_PBO`, `_PAI`, `_F01`, `_F02`) reviewed against the Screen layout guides in `screens/` before activation in SAP
- **Static analysis:** SAP syntax check (SE38 Activate) and extended program check (Program → Check → Extended) run on every code change
- **Test-driven verification:** All 140 QC test cases in the QC Test Plan executed before each UAT round
- **Audit consistency:** `ZBUG_HISTORY` checked after every status-change test to verify log entries
- **Breaking-change validation:** Status constant references (`gc_st_resolved = 'V'`, `gc_st_finaltesting = '6'`) verified against the Status Lifecycle Specification after v5.0 code change

2.3 Training Plan

No formal training was required — all team members had prior SAP ABAP exposure. Specific knowledge gaps were addressed as follows:

Topic	Resource	Member
ALV Grid / Custom Container	SAP Help + reference program `ZPG_BUGTRACKING_MAIN`	`DEV-061`
SmartForms / CL_BCS	SAP Help + SCOT configuration guide	`DEV-061`
Module Pool / Screen Painter	SE51 built-in help + project screen guides in `screens/`	All
10-state lifecycle (v5.0)	Status Lifecycle Specification — v5.0 state machine, transition matrix, auto-assign rules; reviewed before coding Phase F	All

3. Project Deliverables

Internal deliverables (development artifacts):

No.	Deliverable	Description
1	Source Code (6 includes)	ABAP includes `Z_BUG_WS_TOP`, `_F00`, `_PBO`, `_PAI`, `_F01`, `_F02` — v5.0 complete
2	Screen Layout Guides (8 screens)	SE51 layout instructions for Screens 0200/0210/0220/0300/0370/0400/0410/0500 — field lists, Custom Control names, flow logic

3	Database Schema	Table definitions for 6 custom tables (88 fields) — field names, data types, key fields, descriptions
4	QC Test Plan	140 test cases organized in 20 test suites covering all screens, transitions, and edge cases
5	UAT Happy Case Script	64 user-flow test scenarios across 14 workflow categories (A–N) covering all 3 roles
6	Status Lifecycle Specification	v5.0 state machine, role-based transition matrix, auto-assign trigger conditions
7	Phase F Enhancement Guide	Step-by-step deployment instructions F11–F17 for all v5.0 additions

External deliverables (submitted to FPT University):

No.	Deliverable	Description
1	Business Blueprint	SAP Business Blueprint document — compiled Typst PDF
2	Final Report	This document — FPT Capstone Final Report, compiled Typst PDF
3	Test Evidence	Screenshots of UAT Round 1 results and v5.0 regression testing
4	Presentation Slide	Project demo slide deck

4. Responsibility Assignments

No.	Activity	Description	Responsible
1	Database design	Table schemas, domains, data elements, number range	`DEV-089`
2	Function Module design	FM interfaces, permission matrix, auto-assign algorithm	`DEV-089`
3	ABAP coding — core	Includes TOP, F01, F02 (business logic + helpers)	`DEV-089`
4	ABAP coding — UI	Includes PBO, PAI, F00 (screens, ALV, event handler)	`DEV-061`
5	SmartForms / Email	SmartForm `ZBUG_FORM`, `ZBUG_EMAIL_FORM`, CL_BCS integration	`DEV-061`

6	Testing — QC Plan	Write and execute 140 QC test cases	`DEV-118`
7	UAT execution	Execute 43 UAT happy-case scenarios, report defects	`DEV-118`
8	v5.0 Enhancement design	10-state lifecycle, Screen 0370/0410/0210/0220 design	`DEV-089`
9	Documentation	Blueprint, Final Report, screen layout guides, test plans	`DEV-089`
10	Deploy to SAP	SE51 new screens, SE41 GUI statuses, SE93 T-code update	All

5. Project Communications

Channel	Tool	Content	Frequency
Team chat	Messenger / Zalo	Daily progress updates, blockers, quick decisions	Daily
Status report	FPT LMS / email	Weekly progress summary submitted to supervisor	Weekly
Demo / review	SAP GUI screen share	Live demo of features on S40 system	Per milestone
Documentation	Shared documents	Screen guides, test plans, design documents — updated per change	Per change

6. Configuration Management

6.1 Document Management

All project documents are maintained across development phases and carry a version header indicating the last-updated date. The versioning convention used throughout this project is:

- `v4.x` — pre-Phase F documents (initial delivery, Phases A–E)
- `v5.0` — Phase F enhancement documents (current version)

Key document types maintained: Business Blueprint, Final Report, screen layout guides (one per screen), QC Test Plan, UAT Happy Case Script, Database Schema, Status Lifecycle Specification, and Phase F Enhancement Guide.

6.2 Source Code Structure

ABAP source code for the Module Pool `Z_BUG_WORKSPACE_MP` is organized into 6 include programs under SAP package `ZBUGTRACK`. Deployment to SAP is performed manually: open each include in SE38, paste the updated code, check (Ctrl+F2), and activate (Ctrl+F3).

SAP Include	Content	Version
`Z_BUG_WS_TOP`	Global declarations, types, constants for the 10-state lifecycle, ALV/GUI container objects	v5.0
`Z_BUG_WS_F00`	ALV field catalog definitions for 5 grids; `LCL_EVENT_HANDLER` class	v5.0
`Z_BUG_WS_PBO`	Process Before Output modules for all 9 screens	v5.0
`Z_BUG_WS_PAI`	Process After Input modules; all fcode handlers	v5.0
`Z_BUG_WS_F01`	Business logic FORMs: save, change status, auto-assign, email, evidence, history	v5.0
`Z_BUG_WS_F02`	Helper FORMs: F4 search helps, long text API, popup, template download	v5.0

6.3 Tools & Infrastructures

No.	Tool	Version	Purpose
1	SAP GUI	7.70+	Primary SAP front-end for development and testing
2	SE11	—	Data Dictionary: tables, domains, data elements
3	SE38 / SE80	—	ABAP program development and code management
4	SE51	—	Screen Painter: Dynpro screen layout design
5	SE41	—	Menu Painter: GUI Status and Title Bar definition
6	SE93	—	Transaction Maintenance: T-code configuration
7	SMARTFORMS	—	SmartForm designer for PDF and email templates
8	SMW0	—	Web Repository: storage for `.xlsx` evidence templates
9	SCOT	—	Email SMTP configuration for CL_BCS notifications
10	SE16N	—	Table data browser for test data setup and verification
11	Typst	0.11+	Document compilation for Blueprint and Final Report

III. Software Requirement Specification

1. Requirement Overview

1.1 Context Diagram

`ZBUG_WS` is a closed, internal SAP application. All actors interact via SAP GUI using a single T-code entry point. There are no external integrations at the presentation layer; email is the only outbound channel, routed through SAP's internal SCOT/SMTP infrastructure.

System Context Diagram

(Diagram source: `docs/diagrams/context-diagram.mmd`)

Actors:

- **Manager (`DEV-089`):** Full access — create/delete projects, manage users, approve assignments, view dashboard
- **Developer (`DEV-061`):** Receives assigned bugs, updates status (In Progress / Fixed / Pending / Rejected), uploads fix evidence
- **Tester (`DEV-118`):** Creates bugs, uploads bug report evidence, verifies fixes, confirms resolution

1.2 User Requirements

Use Case Diagram (textual representation):

UC Code	Use Case Name	Actor(s)
UC-01	Search and view project list	Manager, Developer, Tester
UC-02	Create / Edit / Delete project	Manager
UC-03	Manage project members (assign role)	Manager
UC-04	View bug list for a project	Manager, Developer, Tester
UC-05	Create bug report	Tester, Manager
UC-06	View bug detail	Manager, Developer, Tester
UC-07	Edit bug information	Manager, Developer (limited), Tester (limited)

UC-08	Change bug status via transition popup	Manager, Developer, Tester (per transition matrix)
UC-09	Delete bug (soft delete)	Manager
UC-10	Upload evidence file	Manager, Developer, Tester (per evidence type)
UC-11	Download evidence template	Manager, Tester
UC-12	Search bugs by multiple criteria	Manager, Developer, Tester
UC-13	View dashboard metrics	Manager
UC-14	Send email notification	System (automatic) + Manager, Developer, Tester (manual)
UC-15	View change history log	Manager, Developer, Tester

1.3 System Functionalities

Screen flow (v5.0):

Screen Navigation Flow (v5.0)

(Diagram source: `docs/diagrams/screen-flow.mmd`)

System roles and screen authorization:

Capability	M	D	T
Access all projects	✓	own	own
Create / Delete project	✓	—	—
Add / Remove project users	✓	—	—
Create bug	✓	—	✓
Delete bug	✓	—	—
Change bug info fields (FNC group: type/priority/severity)	✓	—	✓

Change bug info fields (DEV group: dev note)	✓	✓	—
Change status (via popup 0370)	matrix	matrix	matrix
Upload bug report evidence	✓	—	✓ (reporter)
Upload fix evidence	✓	✓ (assigned)	✓ (config bug)
Download templates from SMW0	✓	—	✓
View dashboard header	✓	✓	✓
Send email manually	✓	✓	✓

2. Functional Specifications

2.1 Bug Management

2.1.1 UC-05 — Create Bug

Use Case ID	**UC-05**
Name	Create Bug
Actor(s)	Tester, Manager
Description	User creates a new bug record linked to the current project context
Pre-condition	User is logged in; a project is selected in Screen 0400; user has Tester or Manager role
Post-condition	New record in `ZBUG_TRACKER` with STATUS='1' (New); auto-assign triggers; email sent

Normal Flow	1. User clicks “Create” on Screen 0200 n 2. Screen 0300 opens in Create mode n 3. User fills TITLE, SAP_MODULE, BUG_TYPE, PRIORITY, SEVERITY n 4. User writes description (Tab 0320) and optional tester note (Tab 0340) n 5. User clicks “Save” n 6. BUG_ID auto-generated (format BUG0000001 via ZNRO_BUG) n 7. Auto-assign phase A runs (New → Assigned or Waiting) n 8. Email sent to assigned developer and manager
Exception Flow	Mandatory fields missing → error message, no save n Auto-assign finds no eligible dev → STATUS = ‘W’ (Waiting)

2.1.2 UC-08 — Change Bug Status

Use Case ID	**UC-08**
Name	Change Bug Status via Popup
Actor(s)	Manager, Developer, Tester (per transition matrix)
Description	User changes bug status through the dedicated popup Screen 0370, which enforces role-based transition rules
Pre-condition	Bug is loaded in Screen 0300 Change mode; user has the correct role for the current status
Post-condition	STATUS updated in `ZBUG_TRACKER`; history record in `ZBUG_HISTORY`; email sent; auto-assign Phase B may trigger
Normal Flow	1. User clicks “Change Status” on Screen 0300 n 2. Popup Screen 0370 opens with current bug info read-only n 3. User selects new status from dropdown (role-filtered options) n 4. User fills required fields (DEVELOPER_ID, TRANS_NOTE, or uploads evidence per matrix) n 5. User clicks “Confirm” n 6. Transition validated; STATUS updated; history logged n 7. If Fixed (5): auto-assign Phase B runs (→ Final Testing 6 or Waiting W)
Exception Flow	Required field missing (e.g., evidence for Fixed, TRANS_NOTE for Resolved) → error message n Transition not allowed for role → popup fields locked, confirm blocked

2.2 Project Management

2.2.1 UC-02 — Create / Edit Project

Use Case ID	**UC-02**

Name	Create / Edit Project
Actor(s)	Manager
Description	Manager creates or edits a project with name, dates, status, and note
Pre-condition	User has Manager role in `ZBUG_USERS`
Post-condition	New/updated record in `ZBUG_PROJECT`; PROJECT_ID auto-generated on create (format PRJ0000001)
Normal Flow	1. Manager clicks “Create Project” on Screen 0400 n 2. Screen 0500 opens in Create mode n 3. Manager fills PROJECT_NAME, START_DATE, END_DATE, PROJECT_MANAGER, PROJECT_STATUS, NOTE n 4. Manager clicks “Save” n 5. PROJECT_ID auto-generated; record saved; mode switches to Change
Exception Flow	PROJECT_NAME empty → error “Project Name is required.” n Set status = 3 (Done) when open bugs exist → error “Cannot set project to Done. N bug(s) not yet Resolved/Closed.”

2.2.2 UC-03 — Manage Project Members

Use Case ID	**UC-03**
Name	Manage Project Members
Actor(s)	Manager
Description	Manager adds or removes users from a project with a specific role (M/D/T)
Pre-condition	Project already saved in `ZBUG_PROJECT`; target user exists in `ZBUG_USERS`
Post-condition	Record in `ZBUG_USER_PROJEC` created or deleted
Exception Flow	User not found in `ZBUG_USERS` → error n Duplicate user → error n Invalid role → error

3. UI Requirements

3.1 Project Search (Screen 0410 — Initial Screen)

The initial screen presented when the user runs T-code `ZBUG_WS`. Filters the project list before displaying it.

UI Requirements:

No.	Field	Description	Required

1	`S_PRJ_ID` (Project ID)	Select-options: filter by project ID, F4 from `ZBUG_PROJECT`	No
2	`S_PRJ_MN` (Project Manager)	Select-options: filter by manager user ID, F4 from `ZBUG_USERS`	No
3	`S_PRJ_ST` (Project Status)	Select-options: filter by status (1/2/3/4), F4 domain values	No

GUI Status `STATUS_0410`: Execute (F8), Back (F3), Exit (Shift+F3), Cancel (F12).

3.2 Bug List (Screen 0200)

Displays all bugs for the selected project (or "My Bugs" filtered by role). Shows a Dashboard Header above the ALV grid.

Dashboard Header fields: Total bugs, by status (New/Assigned/InProgress/Fixed/FinalTesting/Resolved/Waiting/Pending/Rejected), by priority (H/M/L), by module.

ALV columns: BUG_ID (hotspot), TITLE, SAP_MODULE, PRIORITY, STATUS_TEXT, BUG_TYPE, TESTER_ID, DEV_ID, CREATED_AT, ATT_REPORT (hotspot), ATT_FIX (hotspot), ATT_VERIFY (hotspot).

Color coding: Status field colored per SAP color codes (New = blue C510, Assigned = orange C710, In Progress = purple C610, Fixed = green C510, etc.).

3.3 Bug Detail (Screen 0300 — Tab Strip)

Tab strip with 6 subscreens. STATUS field always locked (`STS` screen group); changes only via Screen 0370.

Tab	Subscreen	Content	Editor
1	0310 Bug Info	All bug fields + mini description (`CC_DESC_MINI`)	Standard
2	0320 Description	Full description (Long Text Z001, `CC_DESC`)	`cl_gui_textedit`
3	0330 Dev Note	Developer's fix notes (Long Text Z002, `CC_DEVNOTE`)	`cl_gui_textedit`
4	0340 Tester Note	Tester's verification notes (Long Text Z003, `CC_TSTRNOTE`)	`cl_gui_textedit`

5	0350 Evidence	Evidence upload / download ALV (`CC_EVIDENCE`)	ALV Grid
6	0360 History	Change log from `ZBUG_HISTORY` (`CC_HISTORY`, readonly)	ALV Grid readonly

3.4 Status Transition Popup (Screen 0370 — v5.0 New)

Modal dialog popup triggered by "Change Status" button on Screen 0300.

Read-only fields: BUG_ID, TITLE, REPORTER, CURRENT_STATUS.

Input fields (enabled/locked per current status):

Current Status	NEW_STATUS	DESIGN	FINAL_TESTER	TRANS_NOTE	Upload
1 — New	2, W	Open (→2)	Locked	Locked	Locked
W — Waiting	2, 6	Open	Open (→6)	Locked	Locked
2 — Assigned	3, R	Locked	Locked	Open (→R)	Locked
3 — In Progress	5, 4, R	Locked	Locked	Open	Open (→5)
4 — Pending	2	Open	Locked	Locked	Locked
6 — Final Testing	V, 3	Locked	Locked	Open (→V)	Locked

GUI Status `STATUS_0370`: CONFIRM, UP_TRANS (upload evidence), CANCEL (F12).

4. Non-Functional Requirements

4.1 External Interfaces

- **User interface:** SAP GUI (Dynpro) only — Module Pool screens accessed via T-code `ZBUG_WS`
- **Email:** SAP CL_BCS API with SMTP relay via SCOT. Recipients determined by event type (create, assign, status change, reject)

- **File storage:** Evidence files stored in custom table `ZBUG_EVIDENCE` (RAWSTRING content field). Templates in SMW0 (objects `ZBT_TMPL_01`, `ZBT_TMPL_02`, `ZBT_TMPL_03`)
- **Print:** SmartForms `ZBUG_FORM` (bug detail PDF) and `ZBUG_EMAIL_FORM` (email body HTML)

4.2 Quality Attributes

No.	Attribute	Requirement	Priority
1	Performance	ALV screens load ≤ 3 seconds on S40 with ≤ 1000 bug records	High
2	Security	Role enforcement via `ZBUG_USERS` table check on every screen/ action; no role bypass possible	Critical
3	Auditability	Every status change, assignment, and evidence upload creates a record in `ZBUG_HISTORY`	High
4	Reliability	COMMIT WORK / ROLLBACK WORK used in all save operations; no partial saves	High
5	Usability	Mandatory fields highlighted; confirmation popups for delete/back without save; F4 help on all FK fields	Medium
6	Compatibility	Runs on SAP ABAP 7.70 (SAP_BASIS 770); SAP GUI 7.50+	High

5. Requirement Appendix

5.1 Business Rules

Rule ID	Description
BR-01	Every bug must belong to exactly one project (`PROJECT_ID` is mandatory and locked after creation)
BR-02	BUG_ID is auto-generated via Number Range `ZNRO_BUG` (format: `BUG` + 7-digit NUMC); cannot be manually entered
BR-03	STATUS field is always locked on Screen 0300; status changes are only possible via Screen 0370 (popup)
BR-04	Transitioning to “Fixed (5)” requires at least one evidence file in `ZBUG_EVIDENCE` (COUNT > 0)
BR-05	Transitioning to “Resolved (V)” requires `TRANS_NOTE` to be filled (non-empty confirmation note)
BR-06	Transitioning to “Rejected (R)” or “In Progress→Pending” requires `TRANS_NOTE` (reason for rejection/blocking)

BR-07	Auto-assign selects the developer with the lowest active workload (bugs in statuses 2, 3, 4, 6) AND workload < 5; if none found → STATUS = 'W' (Waiting)
BR-08	Setting project status to "Done (3)" is blocked if any bug in the project has STATUS not in {V, 7, R}
BR-09	Evidence files: format must be `.xlsx`; max size 10 MB; upload after STATUS = 'V' (Resolved) is blocked
BR-10	Manager follows the same transition matrix as other roles (no bypass); all transitions must comply with the v5.0 Status Lifecycle Specification

5.2 Common Requirements

- All screens display a title bar identifying the current screen and context (project name, bug ID)
- Back button (F3) checks for unsaved changes; shows confirmation popup before leaving
- All DELETE operations use soft delete (`IS_DEL = 'X'`); no physical row deletion
- ALV grids support: sort, filter, export to Excel (built-in), sum, find
- All timestamps use SAP system fields: `SY-DATUM`, `SY-UZEIT`, `SY-UNAME`

5.3 Application Messages List

Message Class	Message Content	Type
`ZBUG_MSG`	Bug saved successfully.	S (success)
`ZBUG_MSG`	Please select a bug first.	E (error)
`ZBUG_MSG`	Status transition not allowed for your role.	E
`ZBUG_MSG`	Evidence is required before marking as Fixed.	E
`ZBUG_MSG`	Transition note is required for this status change.	E
`ZBUG_MSG`	Cannot set project to Done. {N} bug(s) not yet Resolved/Closed.	E
`ZBUG_MSG`	User {uid} not found in system.	E
`ZBUG_MSG`	Only managers can create/delete projects.	E
`ZBUG_MSG`	Project saved successfully.	S
`ZBUG_MSG`	Auto-assigned to developer {dev_id}.	S
`ZBUG_MSG`	No available developer found. Status set to Waiting.	W (warning)
`ZBUG_MSG`	Email sent successfully.	S

5.4 Other Requirements

- **Number Range:** `ZNRO_BUG` — interval 01, from 0000000001 to 9999999999, output format `BUG` + NUMC(7)
- **Long Text Object:** `ZBUG_NOTE` (created via SE75) with text IDs Z001 (Description), Z002 (Dev Note), Z003 (Tester Note)
- **SAP Text Object:** text name = BUG_ID (e.g., `BUG0000001`); stored via `SAVE_TEXT` / read via `READ_TEXT`

IV. Software Design Description

1. System Design

1.1 System Architecture

`ZBUG_WS` uses a three-tier architecture running entirely within SAP:

System Architecture (3-Tier)

(Diagram source: `docs/diagrams/system-architecture.mmd`)

The Module Pool calls Function Modules for all business logic — no ABAP logic is coded directly in PBO/PAI modules beyond screen navigation and FM calls.

1.2 Package Diagram

All objects belong to the SAP development package `ZBUGTRACK`:

Package Diagram: ZBUGTRACK

(Diagram source: `docs/diagrams/package-diagram.mmd`)

2. Database Design

Entity Relationship: `ZBUG_TRACKER` (1:N to `ZBUG_HISTORY`, 1:N to `ZBUG_EVIDENCE`) belongs to `ZBUG_PROJECT` (M:N with `ZBUG_USERS` via `ZBUG_USER_PROJEC`).

Table: ZBUG_TRACKER — 29 fields

#	Field	Type	Length	Description
1	`MANDT`	CLNT	3	Client (Key)
2	`BUG_ID`	CHAR	10	Bug ID — Key, auto-generated (`BUG0000001`)
3	`TITLE`	CHAR	100	Bug title — mandatory
4	`DESC_TEXT`	STRING	0	Full description — stored as long text Z001
5	`SAP_MODULE`	CHAR	20	SAP module (MM, SD, FI, CO, etc.)
6	`PRIORITY`	CHAR	1	H=High, M=Medium, L=Low

7	STATUS	CHAR	20	Bug status — 10-state lifecycle (CHAR 20, NOT CHAR 1)
8	BUG_TYPE	CHAR	1	C=Code bug (Dev fixes), F=Config bug (Tester self-fixes)
9	REASONS	STRING	0	Root cause / transition note — STRING type
10	TESTER_ID	CHAR	12	Reporter — auto-filled with SY-UNAME on create
11	VERIFY_TESTER_ID	CHAR	12	Final tester assigned for testing phase (Phase B auto-assign)
12	DEV_ID	CHAR	12	Developer assigned — auto-assign Phase A
13	APPROVED_BY	CHAR	12	Manager who approved (set on close)
14	APPROVED_AT	DATS	8	Approval date
15	CREATED_AT	DATS	8	Creation date — auto SY-DATUM
16	CREATED_TIME	TIMS	6	Creation time — auto SY-UZEIT
17	CLOSED_AT	DATS	8	Closed date (set when status → Resolved/Closed)
18	ATT_REPORT	CHAR	100	Path/name for bug report evidence file
19	ATT_FIX	CHAR	100	Path/name for fix confirmation evidence file
20	ATT_VERIFY	CHAR	100	Path/name for final test confirmation evidence file
21	PROJECT_ID	CHAR	20	FK → ZBUG_PROJECT; locked after creation
22	SEVERITY	CHAR	1	1=Dump, 2=Very High, 3=High, 4=Normal, 5=Minor
23	ERNAM	CHAR	12	Created by — auto SY-UNAME
24	ERDAT	DATS	8	Created date
25	ERZET	TIMS	6	Created time
26	AENAM	CHAR	12	Last changed by
27	AEDAT	DATS	8	Last changed date
28	AEZET	TIMS	6	Last changed time
29	IS_DEL	CHAR	1	Soft delete flag — ‘X’ = deleted

Table: ZBUG_USERS — 12 fields

#	Field	Type	Length	Description
1	MANDT	CLNT	3	Client (Key)
2	USER_ID	CHAR	12	SAP username (Key)
3	ROLE	CHAR	1	M=Manager, D=Developer, T=Tester
4	FULL_NAME	CHAR	50	Full name

5	SAP_MODULE	CHAR	20	SAP module the user specializes in (for auto-assign matching)
6	AVAILABLE_STATUS	CHAR	1	A=Available, B=Busy, L=Leave, W=Working
7	IS_ACTIVE	CHAR	1	X=Active; used to filter in F4 and auto-assign
8	EMAIL	CHAR	100	Email address for CL_BCS notifications
9	AENAM	CHAR	12	Last changed by
10	AEDAT	DATS	8	Last changed date
11	AEZET	TIMS	6	Last changed time
12	IS_DEL	CHAR	1	Soft delete flag

Table: ZBUG_PROJECT — 16 fields

#	Field	Type	Length	Description
1	MANDT	CLNT	3	Client (Key)
2	PROJECT_ID	CHAR	20	Project ID (Key) — auto-generated (PRJ0000001)
3	PROJECT_NAME	CHAR	100	Project name — mandatory
4	DESCRIPTION	CHAR	255	Project description
5	START_DATE	DATS	8	Project start date
6	END_DATE	DATS	8	Project end date
7	PROJECT_MANAGER	CHAR	12	Manager user ID (FK → ZBUG_USERS)
8	PROJECT_STATUS	CHAR	1	1=Opening, 2=In Process, 3=Done, 4=Cancelled
9	NOTE	CHAR	255	Free-text note
10	ERNAM	CHAR	12	Created by
11	ERDAT	DATS	8	Created date
12	ERZET	TIMS	6	Created time
13	AENAM	CHAR	12	Last changed by
14	AEDAT	DATS	8	Last changed date
15	AEZET	TIMS	6	Last changed time
16	IS_DEL	CHAR	1	Soft delete flag

Table: ZBUG_USER_PROJEC — 10 fields

Maps users to projects with a per-project role. Key: MANDT + USER_ID + PROJECT_ID.

Note: table name is ZBUG_USER_PROJEC (truncated at 18 characters — no final ‘T’).

Table: ZBUG_HISTORY — 10 fields

#	Field	Type	Length	Description
1	`MANDT`	CLNT	3	Client (Key)
2	`LOG_ID`	NUMC	10	Auto-generated log ID (Key)
3	`BUG_ID`	CHAR	10	FK → ZBUG_TRACKER
4	`CHANGED_BY`	CHAR	12	Who made the change — SY-UNAME
5	`CHANGED_AT`	DATS	8	Date of change
6	`CHANGED_TIME`	TIMS	6	Time of change
7	`ACTION_TYPE`	CHAR	2	CR=Create, AS=Assign, RS=Reassign, ST=Status Change, UP=Update, DL=Delete, AT=Attachment
8	`OLD_VALUE`	CHAR	100	Previous value
9	`NEW_VALUE`	CHAR	100	New value
10	`REASON`	STRING	0	Reason / transition note — STRING type

3. Detailed Design

3.1 Bug Creation (UC-05)

The creation flow involves the PAI module, the `Z_BUG_CREATE` FM, and the auto-assign engine. The sequence is:

1. User fills fields on Screen 0300 (Create mode) and clicks “Save”
2. PAI calls `Z_BUG_CHECK_PERMISSION` — verifies role = ‘T’ or ‘M’
3. PAI calls `Z_BUG_CREATE`:
   - Calls `NUMBER_GET_NEXT` on `ZNRO_BUG` → generates BUG_ID
   - Sets TESTER_ID = SY-UNAME, ERNAM, ERDAT, ERZET, STATUS = ‘1’
   - If BUG_TYPE = ‘F’ (Config): DEV_ID = SY-UNAME, STATUS = ‘2’
   - Calls `SAVE_TEXT` for description (Text ID Z001)
   - Inserts into `ZBUG_TRACKER`
   - Calls `Z_BUG_LOG_HISTORY` (action = ‘CR’)
   - If BUG_TYPE = ‘C’: calls `Z_BUG_AUTO_ASSIGN` (Phase A)
   - Calls `Z_BUG_SEND_EMAIL` (event = ‘CREATE’ or ‘ASSIGN’)
   - COMMIT WORK

3.2 Status Transition (UC-08)

The status change flow enforces the role-based transition matrix via Screen 0370:

1. User clicks “Change Status” on Screen 0300
2. PAI calls `CALL SCREEN 0370 STARTING AT 5 5` (modal popup)

3. PBO of 0370 loads current bug info and determines which fields to enable/lock based on current STATUS
4. User selects new status, fills required fields, and clicks "Confirm"
5. PAI of 0370 calls `Z_BUG_CHECK_PERMISSION` (action = 'UPDATE_STATUS')
6. Validates transition per the v5.0 role-based transition matrix
7. If STATUS → '5' (Fixed): verifies evidence exists in `ZBUG_EVIDENCE` (COUNT > 0)
8. If STATUS → 'V' or 'R': verifies TRANS_NOTE is non-empty
9. Calls `Z_BUG_UPDATE_STATUS` → updates `ZBUG_TRACKER`, calls `Z_BUG_LOG_HISTORY` (action = 'ST')
10. If STATUS → '5': calls auto-assign Phase B (Fixed → Final Testing or Waiting)
11. Calls `Z_BUG_SEND_EMAIL` (event = 'STATUS_CHANGE')

3.3 Auto-Assign Engine

The auto-assign engine is implemented in FM `Z_BUG_AUTO_ASSIGN` and called at two points:

Phase A (on bug create, BUG_TYPE = 'C'):

- SELECT DEVs from `ZBUG_USER_PROJEC` WHERE project_id = bug.project_id AND role = 'D'
- JOIN `ZBUG_USERS` WHERE sap_module = bug.sap_module AND is_active = 'X' AND is_del ≠ 'X'
- For each dev: COUNT bugs WHERE dev_id = dev AND STATUS IN ('2','3','4','6')
- Select dev with lowest count AND count < 5
- If found: set DEV_ID, STATUS = '2', log history, send email
- If not found: STATUS = 'W', notify Manager

Phase B (on status → Fixed):

- SELECT Testers from `ZBUG_USER_PROJEC` WHERE project_id = bug.project_id AND role = 'T'
- JOIN `ZBUG_USERS` WHERE sap_module = bug.sap_module AND is_active = 'X'
- For each tester: COUNT bugs WHERE verify_tester_id = tester AND STATUS = '6'
- Select tester with lowest count AND count < 5
- If found: set VERIFY_TESTER_ID, STATUS = '6', log history, send email
- If not found: STATUS = 'W', notify Manager

3.4 Role-Based Screen Control (LOOP AT SCREEN)

Dynamic field locking is implemented in MODULE `modify_screen_0300 OUTPUT` using screen groups:

Group	Fields	Lock Condition
`STS`	STATUS	Always locked — change only via popup 0370
`BID`	BUG_ID	Always locked — auto-generated
`PRJ`	PROJECT_ID	Locked once set from project context
`FNC`	BUG_TYPE, PRIORITY, SEVERITY	Locked for Developer role
`TST`	TESTER_ID	Locked for Developer role
`DEV`	DEV_ID, VERIFY_TESTER_ID	Locked for Tester role
`EDT`	All editable fields	Locked in Display mode (input = 0)

The PBO module evaluates `gv_role` (loaded from `ZBUG_USERS` at login) and sets `screen-input = 0` for all fields in the locked groups via `LOOP AT SCREEN ... MODIFY SCREEN`.

V. Software Testing Documentation

1. Scope of Testing

1.1 Target-of-Test Features

The testing scope covers the SAP Bug Tracking Management System (`ZBUG_WS` v5.0), implemented as a Module Pool program (`Z_BUG_WORKSPACE_MP`) on SAP System S40, Client 324. All functional screens, business rules, and role-based access controls are in scope.

In-scope features:

No.	Feature Area	Description
1	Navigation Flow	All screen-to-screen transitions across the 9-screen application (0410 – 0400 – 0200 – 0300 – 0370)
2	Project Search (0410)	New initial screen: 3 filter fields (Project ID, Manager, Status); role-based project visibility
3	Project List (0400)	ALV grid with CRUD operations; soft-delete; Excel template download and upload
4	Project Detail (0500)	Create/Change/Display project; user assignment via Table Control `TC_USERS`
5	Bug List + Dashboard (0200)	ALV grid with real-time dashboard; dual-mode (Project view / My Bugs)
6	Bug Detail (0300)	6-tab strip (Bug Info, Description, Dev Note, Tester Note, Evidence, History); field-level role control via screen groups
7	Status Transition Popup (0370)	10-state lifecycle enforced via popup; role-based transition matrix; required fields per transition
8	Auto-Assign Engine	Phase A: auto-assign Developer on bug creation; Phase B: auto-assign Tester on Fixed transition
9	Bug Search (0210/0220)	Popup search input (6 fields); full-screen results without dashboard
10	Evidence Management	Upload / Download / Delete evidence files; MIME type detection; `EVD_ID` auto-increment
11	Email Notification	BCS API email to assigned Developer and Tester (excluding current user)

12	Template Downloads	SMW0 templates: `Bug_report.xlsx`, `fix_report.xlsx`, `confirm_report.xlsx`
13	Role-Based Access Control	3 roles (Manager, Developer, Tester); field editability via screen groups; button visibility per role
14	Field Validation and Business Rules	Severity/Priority consistency; required fields; unsaved changes detection; error message behavior
15	F4 Search Help	Dropdown helpers on all key fields across screens 0300, 0410, 0370

Out-of-scope:

- Performance load testing (multi-user concurrency scenarios)
- Network-level security penetration testing
- External SMTP server configuration (assumed pre-configured via SOST)
- SAP Basis-level authorization objects (`S_TCODE`, `S_DEVELOP`)

1.2 Test Levels

Testing is conducted at four levels:

1. **Unit Testing** — Individual FORM routines and PAI modules tested in isolation using SAP SE37 and SE38 (e.g., `auto_assign_developer`, `validate_status_transition`, `calculate_dashboard`).
2. **Integration Testing** — Screen-to-screen navigation flows and database read/write interactions verified (e.g., Custom Control lifecycle: create on PBO, free on Back; ALV refresh after status change).
3. **System Testing** — End-to-end workflows executed under all three user roles on a live SAP client (20-suite QC test plan).
4. **Acceptance Testing** — Business-facing UAT happy-case scenarios validated by all three project members.

1.3 Constraints and Assumptions

- All tests execute on SAP System S40, Client 324 (non-production environment).
- Test accounts `DEV-089`, `DEV-061`, `DEV-118` must have correct role assignments in `ZBUG_USERS` prior to testing (verified 11/04/2026).
- Auto-assign tests (TC-09) require additional mock users created by the test data report `Z_BUG_POPULATE_TESTDATA` (20 Developers + 10 Testers across FI/MM/SD/ABAP modules).
- Status migration script (old `status = '6'` Resolved → new `status = 'V'` Resolved) must be executed before regression testing of the v5.0 lifecycle (TC-19.20).

- v5.0 deployment (screens, GUI Statuses, updated ABAP code) must be complete before the full QC test run can proceed.

2. Test Strategy

2.1 Testing Types

No.	Type	Objective	Technique	Completion Criteria
1	Functional Testing	Verify each screen, button, and business rule behaves per the requirements specification	Black-box: provide input, verify expected output; switch between 3 role accounts to test RBAC	All test cases across 20 TC suites pass with no critical (blocking) failures
2	Regression Testing	Confirm that v4.x bug fixes and v5.0 enhancements do not reintroduce previously resolved defects	Re-execute TC-19 (19 cases: 8 v4.x bugs + 11 v5.0 UAT bugs) after deployment	All 19 regression cases pass; no prior defect reappears in the v5.0 build
3	Acceptance Testing (UAT)	Validate that the deployed system meets business workflow requirements from an end-user perspective	Happy-case walkthrough by all 3 roles following the UAT happy-case script (64	All 64 UAT cases pass; no critical blockers remain; all 3 role members sign off

			cases across 14 categories)	

2.2 Test Levels

No.	Level	Description	Test Types Applied
1	Unit	Individual FORM routines tested in isolation — `auto_assign_developer`, `validate_status_transition`, `calculate_dashboard`, `f4_trans_status`; DB read/write operations verified via SE16N	Functional
2	Integration	Screen navigation flows verified end-to-end; Custom Control lifecycle (create/free); ALV refresh after data changes; DB consistency after status transitions	Functional, Regression
3	System	Full end-to-end scenarios under all 3 roles on SAP S40/324; 20-suite QC Test Plan executed systematically by QC lead	Functional, Regression
4	Acceptance	Business-facing happy-case scenarios: project lifecycle (Manager), bug reporting (Tester), bug resolution (Developer), status transitions across the full 10-state lifecycle	Acceptance (UAT)

2.3 Supporting Tools

No.	Tool	Purpose
1	SAP SE38 / SE80	ABAP code editing, syntax check, and activation of all 6 program includes
2	SAP SE37	Unit testing of individual Function Modules: `READ_TEXT`, `SAVE_TEXT`, `F4IF_INT_TABLE_VALUE_REQUEST`, BCS email APIs
3	SAP SE16N	Direct database table inspection — verify DB state after each Create/Update/Delete operation on `ZBUG_TRACKER`, `ZBUG_HISTORY`, `ZBUG_EVIDENCE`
4	SAP SOST	Verify outgoing email queue; confirm BCS API email delivery is queued for assigned users

5	SAP SE51	Screen layout verification; Custom Control placement; flow logic syntax check for all 9 screens
6	SAP SE41	GUI Status and Title Bar inspection; fcode assignment; button visibility validation
7	SAP SE93	Verify T-code `ZBUG_WS` initial screen is set to 0410 (v5.0 requirement)
8	SAP SM50 / SM04	Session and process monitoring during multi-tab switch stress tests (TC-07.15, TC-20.07)

3. Test Plan

3.1 Human Resources

No.	Account	Role	Testing Responsibilities
1	`DEV-118`	QC Lead / Tester	Primary test executor for all 20 TC suites; reports defects with screenshots; validates v5.0 behavior as Tester role (bug creation, evidence upload, Final Testing transition)
2	`DEV-089`	Manager	Co-tester for Manager-role test cases: TC-02 (Project Search), TC-03/04 (Project CRUD), TC-08 Manager transitions, TC-15 RBAC; validates project lifecycle completion rules
3	`DEV-061`	Developer	Co-tester for Developer-role test cases: TC-08 Dev transitions (Assigned → In Progress → Fixed), TC-09 Auto-Assign, TC-12 Evidence (Upload Fix, Download)

3.2 Test Environment

No.	Component	Details	Status
1	SAP System	S40 — dedicated internal development and test system	Ready
2	SAP Client	324 (test client; Table Maintenance Generator available via SM30)	Ready
3	ABAP Version	7.70 (SAP_BASIS 770) — supports inline declarations, SWITCH expression, @ host variables, string templates	Verified
4	Development Package	`ZBUGTRACK` — contains all program objects (screens, includes, GUI Statuses)	Active
5	Network	EBS_SAP internal network	Ready

6	Test Accounts	DEV-089 (Manager), DEV-061 (Developer), DEV-118 (Tester) — roles verified in ZBUG_USERS on 11/04/2026	Verified
7	Auto-Assign Test Data	20 mock Developers + 10 mock Testers across FI/MM/SD/ABAP modules; created by report Z_BUG_POPULATE_TESTDATA; assigned to a test project via ZBUG_USER_PROJEC	Pending
8	SMW0 Templates	3 Excel templates uploaded: ZBT_TMPL_01 (Bug_report.xlsx), ZBT_TMPL_02 (fix_report.xlsx), ZBT_TMPL_03 (confirm_report.xlsx)	Pending

3.3 Test Milestones

No.	Milestone	Start Date	End Date	Deliverable
1	Test Environment Setup	01/04/2026	11/04/2026	ZBUG_USERS role assignments verified; test accounts confirmed; SE16N screenshots captured
2	UAT Round 1 (v4.2 build)	11/04/2026	13/04/2026	64 UAT happy cases executed; 11 defects identified and documented in the v5.0 Defect Analysis document
3	v5.0 Bug Analysis & Design	13/04/2026	14/04/2026	Root-cause analysis and fix proposals for all 11 UAT defects; Phase F v5.0 architecture finalized
4	v5.0 Code Development (F10)	14/04/2026	16/04/2026	All 6 ABAP includes (Z_BUG_WS_TOP through Z_BUG_WS_F02) updated to v5.0; verified complete on 16/04/2026
5	v5.0 Deployment to SAP (F11–F17)	Post 16/04/2026	TBD	4 new screens (SE51), 4 GUI Statuses + 4 Title Bars (SE41), SE93 initial screen update, ZBUG_EVIDENCE table, migration script, SMW0 templates
6	Full QC Test Run — 20 Suites	After Deployment	TBD	Approximately 210 test cases executed; defect report; pass/fail statistics per suite
7	UAT Round 2 (v5.0 build)	After QC Run	TBD	64 UAT happy cases re-executed on deployed v5.0 system; all 3 members sign off; final acceptance

4. Test Cases

Test cases are organized into 20 suites (TC-01 to TC-20), totaling approximately 210 individual cases. Detailed test case steps and expected results are maintained in the QC Test Plan document. UAT happy cases (64 cases across 14 workflow categories, A–N) are documented in the UAT Happy Case Script.

Test Suite Summary:

No.	TC ID	Suite Name	Coverage	Cases
1	TC-01	Navigation Flow	All screen transitions: 0410→0400→0200→0300→0370; Back/Exit/ Cancel from every screen	20
2	TC-02	Screen 0410 – Project Search	Filter by Project ID / Manager / Status; role-based project visibility; F4 help on 3 fields	14
3	TC-03	Screen 0400 – Project List	ALV display; Project CRUD; soft-delete; Refresh; non-Manager restriction	12
4	TC-04	Screen 0500 – Project Detail	Create/Change/Display; user assignment Table Control; F4 calendar; non-Manager blocking	25
5	TC-05	Screen 0200 – Bug List + Dashboard	Dual-mode (Project / My Bugs); dashboard accuracy; Bug CRUD; ALV row coloring	18
6	TC-06	Screen 0300 – Bug Detail	Create/Change/Display; STATUS always locked; display text mapping for all 10 states	18
7	TC-07	Tab Strip and Subscreens	All 6 tabs; data persistence on switch; role-specific editor read-only; crash prevention	15
8	TC-08	Status Transition (10-State + Popup 0370)	All valid transitions; blocked/invalid transitions; required fields per transition; history logging	30
9	TC-09	Auto-Assign System	Phase A (Developer): module match + workload < 5; Phase B (Tester): same logic on Fixed transition	9
10	TC-10	Bug Search (Screens 0210/0220)	All 6 search fields; wildcard title; combined filters; results ALV; back navigation	15
11	TC-11	Dashboard Metrics	Count accuracy per status / priority; sum validation (status sum = Total); real-time refresh	12

12	TC-12	Evidence Management	Upload (generic / report / fix); download; delete; MIME detection; `EVD_ID` auto-increment; popup upload	12
13	TC-13	Email Notification	BCS API send; no-recipient handling; current-user exclusion from recipient list; SOST verification	4
14	TC-14	Template Download and Upload	3 SMW0 template downloads; Excel project upload; duplicate/invalid record handling	11
15	TC-15	Role-Based Access Control	Field editability by role (FNC/DEV/TST groups); button visibility per role (Create/Delete/Upload)	16
16	TC-16	Field Validation and Business Rules	Severity/Priority consistency rule; required fields; long text persistence; error message does not lock fields	9
17	TC-17	Unsaved Changes Detection	Save/Discard/Cancel popup; snapshot comparison; mini editor text sync before compare	9
18	TC-18	F4 Search Help	All F4 helpers on screens 0300 (9 fields), 0410 (3 fields), 0370 (3 fields); value fill-back	16
19	TC-19	Regression – Fixed Bugs v4.x + v5.0	8 v4.x bugs + 11 v5.0 UAT bugs: verify none reappear in the v5.0 build	19
20	TC-20	Edge Cases and Boundary Testing	Empty DB; max-length fields; large file upload; memory leak on Back; unregistered user handling	20
		Total (estimated)		**210**

5. Test Reports

5.1 UAT Round 1 — Phase E Results (11–13 April 2026)

UAT Round 1 was conducted in Phase E on the v4.2 deployed build (SAP System S40, Client 324). All three project members participated as their designated roles, following the UAT happy-case script (64 cases across 14 categories).

Summary:

Total Cases	Passed	Failed	Blocked	Defects Found
64	53	11	0	11

Defects Found in UAT Round 1:

No.	Bug ID	Defect Description	Severity
1	UAT-01	Tab switch between Description/Dev Note/Tester Note tabs triggers `CALL_FUNCTION_CONFLICT_TYPE` short dump (Custom Control not freed before re-creation)	Critical
2	UAT-02	`DESC_TEXT` (STRING type) placed on screen 0310 layout causes screen rendering failure – STRING fields not supported on Dynpro layouts	High
3	UAT-03	Bug Info tab (0310) missing fields: `SAP_MODULE`, `DEV_ID`, `VERIFY_TESTER_ID`, `ATT_REPORT`, `ATT_FIX` – not placed on screen layout	High
4	UAT-04	Remove User button deletes without checking current selected row in Table Control `TC_USERS` – removes wrong or no row	Medium
5	UAT-05	Create Bug: incorrect default values; `SAP_MODULE` F4 help not registered; description text lost after save and reopen	High
6	UAT-06	F4 help on `SAP_MODULE` field shows no popup – POV flow logic entry missing for that field on screen 0310	Medium
7	UAT-07	Validation error message using `TYPE 'E'` locks all screen fields – user cannot correct input without navigating away	High
8	UAT-08	Description text disappears after reopening a saved bug – long text not re-loaded correctly on PBO when bug is re-entered	High
9	UAT-09	Backward status transition allowed (e.g., Fixed → In Progress) – no transition matrix enforced in v4.2	Critical
10	UAT-10	No evidence file check before marking bug as Fixed – transition proceeds even with empty `ZBUG_EVIDENCE`	High
11	UAT-11	Manager can bypass transition matrix – direct status assignment in `save_bug_detail` overwrites any status without validation	Critical

Resolution: All 11 defects were root-cause analyzed and resolved in Phase F v5.0. Fixes are incorporated into the 6 ABAP includes (`Z_BUG_WS_TOP` through `Z_BUG_WS_F02`), all updated to v5.0 (F10 COMPLETE as of 16/04/2026). UAT Round 2 verification is scheduled after Phase F deployment (Steps F11–F17).

5.2 QC Full Test Run — Phase F (Planned)

The full 20-suite QC test plan (210 cases) is scheduled for Phase F, Step F14, after successful deployment of all v5.0 components to SAP System S40. Priority execution order: TC-19

(Regression) → TC-01 (Navigation) → TC-08 (Status Transition) → TC-09 (Auto-Assign) → TC-06 (Bug Detail) → TC-15 (RBAC) → TC-11 (Dashboard) → TC-10 (Bug Search) → TC-02 (Project Search) → remaining suites.

Target metrics for QC completion:

Total TCs	Target Pass	Max Failures	Blocked	Pass Rate Target
210	>= 200	< 10	0	>= 95%

VI. Release Package & User Guides

1. Deliverable Package

All deliverables below pertain to version v5.0 of the SAP Bug Tracking Management System (`ZBUG_WS`), implemented as Module Pool program `Z_BUG_WORKSPACE_MP` in SAP package `ZBUGTRACK`.

No.	Deliverable Item	Description	Version
1	ABAP Include `Z_BUG_WS_TOP`	Global declarations, types, constants for the 10-state lifecycle (`gc_st_new` through `gc_st_resolved`), all ALV/GUI container objects, dashboard metric variables, Screen 0370/0410/0210/0220 field variables	v5.0
2	ABAP Include `Z_BUG_WS_F00`	ALV field catalog definitions for 5 ALV grids; `LCL_EVENT_HANDLER` class implementing hotspot-click and double-click events for Bug List and Project List	v5.0
3	ABAP Include `Z_BUG_WS_PBO`	Process Before Output modules for all 9 screens; dashboard calculation call in `status_0200`; screen group logic via `LOOP AT SCREEN` for role-based field control	v5.0
4	ABAP Include `Z_BUG_WS_PAI`	Process After Input modules; all fcode handlers; status transition popup call (`CALL SCREEN 0370`); bug search engine trigger (`CALL SCREEN 0210`)	v5.0
5	ABAP Include `Z_BUG_WS_F01`	Business logic FORMs: `save_bug_detail`, `save_project_detail`, `change_bug_status`, `auto_assign_developer` (Phase A), `auto_assign_tester` (Phase B), `upload_evidence_file`, `send_email_bcs`, `log_history`, `calculate_dashboard`	v5.0
6	ABAP Include `Z_BUG_WS_F02`	Helper FORMs: 10 F4 search help routines, `load_long_text` / `save_long_text` (long text API), `download_smw0_template` wrapper, `upload_excel_projects` parser	v5.0
7	DB Table `ZBUG_TRACKER`	Primary bug tracking table — 29 fields. Domain `zde_bug_status` is CHAR 20 (not CHAR 1); STATUS field stores 10-state lifecycle codes (1, 2, 3, 4, 5, 6, 7, R, V, W)	v5.0

8	DB Table `ZBUG_PROJECT`	Project master — 16 fields including soft-delete flag `IS_DEL` and full audit trail (ERNAM, ERDAT, AENAM, AEDAT)	v5.0
9	DB Table `ZBUG_USERS`	User registry — 12 fields including `ROLE` (M/D/T), `SAP_MODULE`, `IS_ACTIVE`, `IS_DEL`, and `EMAIL` for notification	v5.0
10	DB Table `ZBUG_USER_PROJEC`	User-to-project M:N assignment — 10 fields; `ROLE` column per project; basis for auto-assign filtering and role-based project visibility	v5.0
11	DB Table `ZBUG_HISTORY`	Change log — 10 fields; records all status transitions and field updates with `OLD_STATUS`, `NEW_STATUS`, `ACTION` (ST/CR/UP), `REASON` (STRING), and timestamps	v5.0
12	DB Table `ZBUG_EVIDENCE`	Binary file storage — 11 fields; `CONTENT` as RAWSTRING; `MIME_TYPE` (CHAR 100), `FILE_SIZE` (INT4), `FILE_NAME` (CHAR 255)	v5.0
13	Screen Guides (8 screens)	SE51 layout guides under `screens/` directory for screens 0200, 0210, 0220, 0300 (+ 6 subscreens 0310–0360), 0370, 0400, 0410, 0500 — field lists, Custom Control names, flow logic	v5.0
14	QC Test Plan + UAT Script	QC Test Plan: 20 suites (TC-01 to TC-20), approximately 210 individual test cases covering all screens, transitions, edge cases, and RBAC. UAT Happy Case Script: 64 cases across 14 workflow categories (A–N) for Manager, Developer, and Tester roles	v5.0
15	Final Report Document	This document (FPT Capstone Final Report) — covers Introduction, Project Management, Requirements, Design, Testing, and Release sections	v5.0
16	SMW0 Templates (3 files)	`ZBT_TMPL_01` → `Bug_report.xlsx` (Tester bug report); `ZBT_TMPL_02` → `fix_report.xlsx` (Developer fix evidence); `ZBT_TMPL_03` → `confirm_report.xlsx` (Final Tester sign-off)	v5.0
17	Test Data Report	`Z_BUG_POPULATE_TESTDATA` — SE38 executable report creating 20 mock Developers + 10 mock Testers across FI/MM/SD/ABAP modules with project assignments for auto-assign algorithm testing	v5.0

18	Status Migration Script	One-time ABAP migration: updates all records with `STATUS = '6'` (Resolved in v4.x) to `STATUS = 'V'` (Resolved in v5.0) via `UPDATE zbug_tracker SET status = 'V' WHERE status = '6' AND is_del <> 'X'` followed by `COMMIT WORK`	v5.0

2. Installation Guides

2.1 System Requirements

No.	Component	Requirement	Version / Value
1	SAP System	SAP ERP with full ABAP Workbench (SE11, SE38/ SE80, SE51, SE41, SE93, SE37, SM30 available)	S40
2	SAP Basis	SAP_BASIS 770 or higher — required for ABAP 7.70 syntax (inline `DATA`, `SWITCH`, `CONV`, `@` host variables, string template `|...|`)	770+
3	SAP Client	Dedicated development/test client with Table Maintenance Generator accessible for `ZBUG_USERS` via SM30	Client 324
4	Email (SOST)	SOST / BCS API configured with valid SMTP profile for outgoing email notifications to Developer and Tester	Pre-configured
5	Web Repository (SMW0)	SMW0 repository accessible; `ZBT_TMPL_01`, `ZBT_TMPL_02`, `ZBT_TMPL_03` objects created and Excel files uploaded	Active
6	Number Range Objects	`ZNR_BUGS` (10-digit Bug ID, format `BUG0000001`) and `ZNR_PROJECTS` (10-digit Project ID, format `PRJ0000001`) must exist in `SNRO`	Pre-configured
7	Text Object	`ZBUG` text object registered in `SE75` for `READ_TEXT`/ `SAVE_TEXT` long text storage (Description, Dev Note, Tester Note)	Pre-configured
8	Development Package	`ZBUGTRACK` package must exist in SE80; all program objects assigned to this package	Active

2.2 Installation Steps

The following steps deploy `Z_BUG_WORKSPACE_MP` v5.0 into the SAP system. Execute in the listed order. Steps F11–F17 correspond to Phase F deployment tasks.

Step F11 — Create 4 New Screens in SE51:

1. Open transaction **SE51**. Set Program = `Z_BUG_WORKSPACE_MP`.
2. **Screen 0410** (Normal screen): Create screen, set short description "Project Search". In the layout, add 3 input fields: `S_PRJ_ID` (label "Project ID"), `S_PRJ_MN` (label "Manager"), `S_PRJ_ST` (label "Status"). In Flow Logic, add: PBO → `MODULE status_0410 OUTPUT`; PAI → `MODULE user_command_0410 INPUT`; POV → F4 modules for all 3 fields. Activate.
3. **Screen 0370** (Modal Dialog Box, 80 columns × 20 rows): Add read-only display fields for `gv_trans_bug_id`, `gv_trans_title`, `gv_trans_reporter`, `gv_trans_cur_st_text`. Add input fields: `gv_trans_new_status` (label "New Status"), `gv_trans_dev_id` (label "Developer"), `gv_trans_ftester_id` (label "Final Tester"). Add Custom Control named `CC_TRANS_NOTE`. Flow Logic: PBO → `status_0370`, `init_trans_popup`; PAI → `user_command_0370`; POV → F4 modules. Activate.
4. **Screen 0210** (Modal Dialog Box, 70 columns × 15 rows): Add 7 input fields: `s_bug_id`, `s_title`, `s_status`, `s_prio`, `s_mod`, `s_reporter`, `s_dev`. Flow Logic: PBO → `status_0210`; PAI → `user_command_0210`; POV → F4 modules. Activate.
5. **Screen 0220** (Normal screen): Add Custom Control named `CC_SEARCH_RESULTS`. Flow Logic: PBO → `status_0220 OUTPUT`, `display_search_alv OUTPUT`; PAI → `user_command_0220 INPUT`. Activate.

Step F12 — Create GUI Statuses and Title Bars in SE41:

1. Open transaction **SE41**. Set Program = `Z_BUG_WORKSPACE_MP`.
2. Create GUI Status `STATUS_0410`: Application Toolbar with buttons Execute (fcode `EXECUTE`, F8), Back (fcode `BACK`, F3), Exit (fcode `EXIT`, Shift+F3), Cancel (fcode `CANCEL`, F12). Create Title Bar `T_0410` with text "Project Search". Activate both.
3. Create GUI Status `STATUS_0370`: Application Toolbar with Confirm (fcode `CONFIRM`), Upload Evidence (fcode `UP_TRANS`), Cancel (fcode `CANCEL`, F12). Create Title Bar `T_0370` with text "Change Bug Status". Activate.
4. Create GUI Status `STATUS_0210`: Application Toolbar with Execute (fcode `EXECUTE`, F8) and Cancel (fcode `CANCEL`, F12). Create Title Bar `T_0210` with text "Bug Search". Activate.
5. Create GUI Status `STATUS_0220`: Application Toolbar with Back (fcode `BACK`, F3), Exit (fcode `EXIT`, Shift+F3), Cancel (fcode `CANCEL`, F12). Create Title Bar `T_0220` with text "Search Results". Activate.
6. Update existing GUI Status `STATUS_0200`: add button Search (fcode `SEARCH`) to the Application Toolbar. Re-activate.

Step F13 — Copy v5.0 ABAP Code into SAP:

1. Open transaction **SE80** or **SE38**.
2. For each of the 6 includes listed below, open the include program in SE38, paste the v5.0 source code, then check (Ctrl+F2) and activate (Ctrl+F3):

SAP Include Program	Content Summary
`Z_BUG_WS_TOP`	Global declarations, types, constants, ALV objects
`Z_BUG_WS_F00`	ALV field catalog, `LCL_EVENT_HANDLER` class
`Z_BUG_WS_PBO`	Process Before Output modules (all 9 screens)
`Z_BUG_WS_PAI`	Process After Input modules, all fcode handlers
`Z_BUG_WS_F01`	Business logic FORMs: save, status change, auto-assign, email
`Z_BUG_WS_F02`	Helper FORMs: F4, long text API, popup, template download

1. After all 6 includes are copied, open the main program `Z_BUG_WORKSPACE_MP` in SE80 and perform a mass activation (select all objects → Activate). Resolve any syntax errors before proceeding.

Step F14 — Update T-code Initial Screen in SE93:

1. Open transaction **SE93**. Enter `ZBUG_WS` → Change.
2. Under "Default values", change field "Screen number" from `0400` to `0410`.
3. Save and activate. Run `/nZBUG_WS` to confirm Screen 0410 appears as the first screen.

Step F15 — Create `ZBUG_EVIDENCE` Table (if not already present):

1. Open transaction **SE11**. Create database table `ZBUG_EVIDENCE` with 11 fields as listed below. Key fields: `CLIENT` (MANDT, CLNT 3, Key), `EVD_ID` (CHAR 10, Key). The `CONTENT` field must use type `RAWSTRING` (binary blob). Additional fields: `BUG_ID` (CHAR 10), `FILE_NAME` (CHAR 255), `MIME_TYPE` (CHAR 100), `FILE_SIZE` (INT4), `ERNAM` (CHAR 12), `ERDAT` (DATS 8), `ERZET` (TIMS 6), `EVD_TYPE` (CHAR 1 — R=Report, F=Fix, V=Verify). Activate the table and generate the Table Maintenance Generator if needed.

Step F16 — Run Status Migration Script:

1. Open **SE38**. Create a temporary report and run the following ABAP:

```
UPDATE zbug_tracker SET status = 'V'
  WHERE status = '6' AND is_del <> 'X'.
COMMIT WORK.
WRITE: / 'Migrated', sy-dbcnt, 'records from status 6 to V.'.
```

1. Verify via **SE16N** on `ZBUG_TRACKER` that no records remain with `STATUS = '6'`. After verification, delete the temporary report.

Step F17 — Upload SMW0 Templates:

1. Open transaction **SMW0**. Navigate to "Binary data for WebRFC applications".
2. Upload the 3 Excel template files to the corresponding objects:
   - Object `ZBT_TMPL_01` ← `Bug_report.xlsx` (Tester bug report template)
   - Object `ZBT_TMPL_02` ← `fix_report.xlsx` (Developer fix evidence template)
   - Object `ZBT_TMPL_03` ← `confirm_report.xlsx` (Final Tester sign-off template)
3. Save and activate all 3 objects.

Deployment Verification: After completing all steps, run `/nZBUG_WS`. Confirm Screen 0410 (Project Search) appears first. Press Execute → Screen 0400 shows filtered project list. Click a project → Screen 0200 shows bug list with Dashboard header. Select a bug in Change mode → click "Change Status" → Screen 0370 popup opens with correct bug info and status dropdown.

3. User Manual

3.1 Overview

The SAP Bug Tracking Management System (`ZBUG_WS`) is a centralized defect tracking application built natively on SAP ERP using ABAP Module Pool programming. The system supports three user roles — **Manager**, **Developer**, and **Tester** — each with distinct permissions enforced at the screen group and fcode level.

System entry: Transaction code `ZBUG_WS` → Screen 0410 (Project Search)

Screen map:

Screen	Name	Purpose
0410	Project Search	Initial screen: filter accessible projects before entering the list (New in v5.0)
0400	Project List	ALV grid of projects; create/change/delete projects; launch My Bugs
0200	Bug List + Dashboard	Bug listing for a selected project with real-time status / priority / module metrics

0300	Bug Detail	Create/view/edit a bug; 6-tab strip: Bug Info, Description, Dev Note, Tester Note, Evidence, History
0370	Status Transition Popup	Change bug status via 10-state matrix; validates role and required fields (New in v5.0)
0500	Project Detail	Manage project metadata and assign/remove users from a project (Manager only)
0210/0220	Bug Search	Search bugs by keyword, status, priority, module, or reporter across the current project

10-State Bug Lifecycle (v5.0):

10-State Bug Lifecycle (v5.0)

(Diagram source: `docs/diagrams/bug-lifecycle.mmd`)

Note: `Closed (7)` is a legacy terminal state retained for backward compatibility.

3.2 Workflow 1 — Bug Reporting (Tester Role)

Purpose: A Tester discovers a defect during SAP system testing, logs it in the system, and attaches supporting evidence.

Prerequisites: Logged in with a Tester account (role `T` in `ZBUG_USERS`); at least one project exists with this Tester assigned.

Steps:

1. Run transaction `/nZBUG_WS`. Screen 0410 (Project Search) appears with three filter fields.
2. Press **Execute** (F8) to list all accessible projects (or enter a Project ID filter first). Screen 0400 shows the project list.
3. Double-click the target project row. Screen 0200 opens, showing all bugs for that project and the real-time Dashboard header.
4. Click the **Create** button in the toolbar. Screen 0300 opens in Create mode.
5. On the **Bug Info** tab, fill in the required fields:
   - **Title**: concise defect description (required, max 100 characters)
   - **SAP Module**: press F4 to select from FI / MM / SD / ABAP / Basis / PP / HR / QM
   - **Priority**: press F4 to select H (High) / M (Medium) / L (Low)
   - **Severity**: press F4 to select 1 (Dump/Critical) through 5 (Minor)
   - **Bug Type**: press F4 to select 1 (Functional) through 5 (Security)

- **Project ID**: pre-filled and locked from the project context (cannot be changed)

6. Switch to the **Description** tab: enter a full description of the defect using the text editor.
7. Switch to the **Evidence** tab: click **Upload Evidence** → select a screenshot or log file from the local PC. The file is stored in `ZBUG_EVIDENCE` and appears as a new row in the Evidence table.
8. Click **Save**. The system auto-generates a `BUG_ID` (format `BUG0000001`) and immediately runs the auto-assign engine:
   - **If a matching Developer is found** (same SAP Module, active in the project, workload < 5 bugs): `DEV_ID` is filled, status becomes **Assigned (2)**.
   - **If no matching Developer is available**: status becomes **Waiting (W)** and a notification message is shown.
9. The bug now appears in the Bug List on Screen 0200. Dashboard counters update accordingly.

3.3 Workflow 2 — Bug Resolution (Developer Role)

Purpose: A Developer picks up an assigned bug, investigates, applies a fix, uploads fix evidence, and marks the bug as Fixed.

Prerequisites: Logged in with a Developer account (role `D` in `ZBUG_USERS`); at least one bug is assigned with `DEV_ID` matching the current user.

Steps:

1. Run `/nZBUG_WS` → Execute → click the relevant project. On Screen 0200, find the assigned bug (status = Assigned).
   - Alternatively, click **My Bugs** on the Screen 0400 toolbar to see only bugs where `DEV_ID = current user`.
2. Select the bug → click **Change**. Screen 0300 opens in Change mode.
3. Click the **Change Status** button. Popup Screen 0370 opens displaying the bug's current information:
   - Read-only fields: Bug ID, Title, Reporter, Current Status ("Assigned")
   - Dropdown: **New Status** — press F4, select "In Progress (3)"
   - Click **Confirm**. Status changes to In Progress.
4. Investigate the defect. Document findings and the fix approach in the **Dev Note** tab (text editor).
5. Switch to the **Evidence** tab and click **Upload Fix** → select the fix proof file (e.g., `fix_report.xlsx`). At least one evidence file must exist before the Fixed transition is allowed.
6. Click **Change Status** again. In popup 0370:

- Press F4 on New Status → select "Fixed (5)"
- Click **Confirm**. The system validates that at least one evidence file exists in `ZBUG_EVIDENCE`. On success:
  - Status changes to Fixed (5).
  - Auto-Assign Phase B triggers immediately: the system searches for a Tester with matching `SAP_MODULE` in the project and workload < 5 active Final Testing bugs.
  - **If a Tester is found**: `VERIFY_TESTER_ID` is set, status advances to **Final Testing (6)**.
  - **If no Tester is available**: status becomes **Waiting (W)**.

7. A history record is written to `ZBUG_HISTORY` for the Fixed transition. The bug responsibility transfers to the assigned Tester.

3.4 Workflow 3 — Project Management (Manager Role)

Purpose: A Manager creates a new project, assigns the development team, monitors bug progress using the Dashboard, and closes the project when all bugs are resolved.

Steps — Create Project and Assign Team:

1. Run `/nZBUG_WS` → Execute (F8) on Screen 0410 to view all projects. Screen 0400 opens.
2. Click **Create Project** in the toolbar. Screen 0500 opens in Create mode.
3. Fill in the project fields:
   - **Project Name**: required
   - **Start Date** / **End Date**: press F4 for a calendar picker
   - **Project Status**: defaults to "1 — Opening"
   - **Project Manager**: pre-filled with the current user's ID
4. Click **Save**. A `PROJECT_ID` is auto-generated (format `PRJ0000001`). Mode switches to Change.
5. In the **User Assignment** Table Control at the bottom of Screen 0500:
   - Click **Add User** → enter User ID (press F4 to browse `ZBUG_USERS`) and Role (M/D/T)
   - Repeat for each team member (Developers, Testers, co-Managers as needed)
   - Save after each addition
6. Click **Back** to return to Screen 0400. The new project appears in the list.

Steps — Monitor Bug Progress:

1. On Screen 0400, click the project row to open Screen 0200 (Bug List + Dashboard).
2. The Dashboard header at the top of Screen 0200 displays real-time metrics:
   - **Total Bugs**: total bug count in this project
   - **By Status**: count per lifecycle state (New / Assigned / In Progress / Fixed / Final Testing / Resolved / Waiting / Rejected)

- **By Priority**: High / Medium / Low counts
- **By Module**: FI / MM / SD / ABAP / Basis counts

3. Click **Refresh** in the toolbar at any time to reload bug data and recalculate the Dashboard.
4. Click **Search** to open the Bug Search popup (Screen 0210): enter up to 6 search criteria (Bug ID, title keyword, status, priority, module, reporter). Screen 0220 shows filtered results without a dashboard.
5. A Manager can perform manual status transitions on any bug: select bug → Change → **Change Status** → Screen 0370 popup → select any valid transition per the matrix → Confirm.

Steps — Close Project:

1. On Screen 0400, select the project → **Change**. Screen 0500 opens.
2. In the **Project Status** field, select "3 — Done".
3. Click **Save**. The system validates that all bugs in this project are in terminal states (Resolved V, Rejected R, or Closed 7).
   - **If open bugs exist**: message "Cannot set project to Done. N bug(s) not yet Resolved/ Closed." Save is rejected.
   - **If all bugs are terminal**: project status changes to Done; save succeeds.
4. Return to Screen 0410 and filter by Status = "3 — Done" to confirm the project is closed.